Ella Tech Backend Engineering

System Architecture: Test Automation, DevOps & Quality Reporting

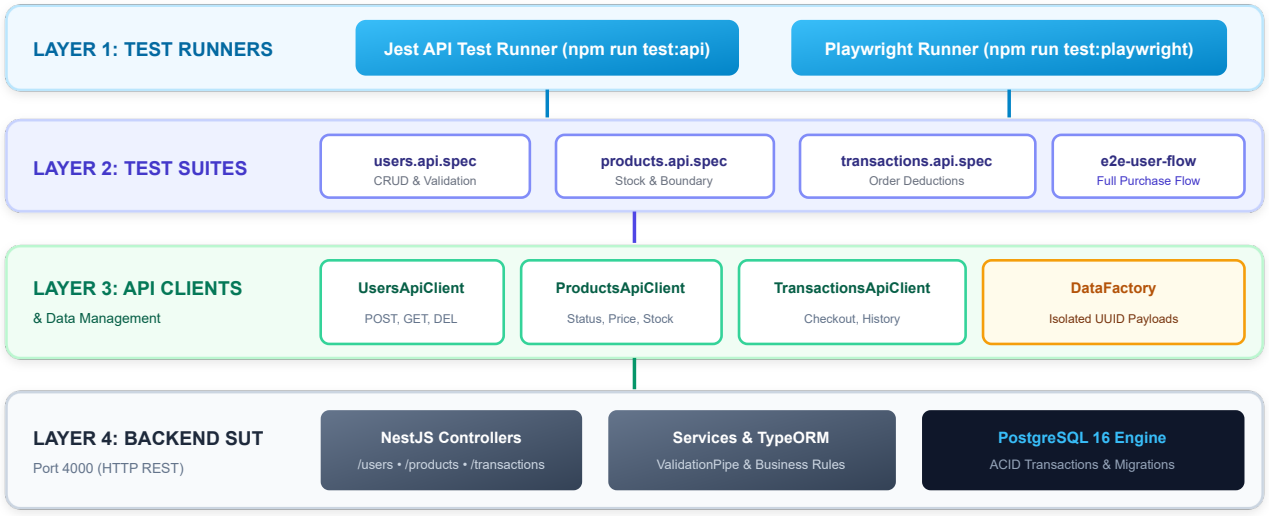
Project: Ella API Assessment
Engine: NestJS 11 • TypeORM • PostgreSQL 16
Author: QA & DevOps Engineering

Executive Summary: This architectural blueprint details the end-to-end quality assurance, containerization, and continuous delivery ecosystem designed for the Ella Tech e-commerce platform. It provides visual schematics and technical breakdowns for the **Automated API Testing Framework**, the **CI/CD GitHub Actions Pipeline**, and the **Zero-Dependency Interactive HTML Reporting Engine**.

1. Automated API Testing Framework Architecture

The automated testing framework employs a layered, object-oriented client architecture that cleanly decouples test assertions from underlying HTTP transport logic, ensuring high maintainability, reusable payloads, and resilient end-to-end execution.

FIGURE 1.1: AUTOMATED TESTING LAYERED ARCHITECTURE & DATA FLOW — JEST & PLAYWRIGHT DUAL-ENGINE



FLOW Critical E2E User Purchase Flow

The framework verifies the entire e-commerce lifecycle seamlessly:

- **Step 1:** Register fresh buyer with unique timestamped email via `POST /users`.
- **Step 2:** Create active product with known stock (e.g., 50 units) via `POST /products`.
- **Step 3:** Buyer submits order for 3 items via `POST /transactions`.
- **Step 4:** Validate `201 Created`, calculate total price, verify remaining stock is 47.
- **Step 5:** Boundary check: Attempting to order 48 items immediately rejects with `400 Bad Request`.

DESIGN Core Framework Architectural Patterns

Key design principles applied throughout the automation code:

- **Fluent Base Client:** Centralized Axios/Supertest wrappers with automatic status-code assertions and response unnesting.
- **Isolated Data Generation:** `DataFactory` prevents collisions in parallel test runs without wiping shared databases.
- **Dual Execution Options:** Jest for lightweight CI API testing + Playwright for unified cross-browser/API orchestration.
- **Automatic Health-Gate:** Pre-test suite checks `GET /health` before firing heavy suites.

DevOps CI/CD & Infrastructure

Automated Quality Gates, Containerization & Deployment Pipeline

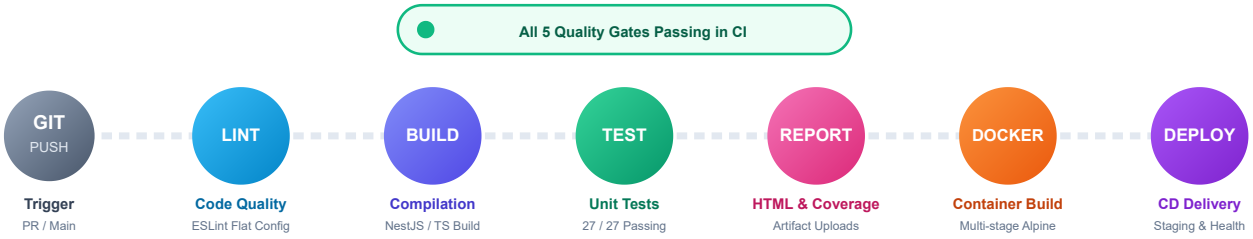
Pipeline: GitHub Actions (.github/workflows/ci.yml)
Runtime: Node 22-Alpine • Multi-Stage Docker

2. CI/CD Pipeline Architecture & Delivery Gates

Every code push and pull request executes through a strictly sequenced, fail-fast CI workflow. Quality gates ensure broken builds, linting regressions, or unit test failures block merges before deployment.

FIGURE 2.1: CONTINUOUS INTEGRATION & CONTINUOUS DEPLOYMENT PIPELINE LIFECYCLE

FAIL-FAST ARCHITECTURE



DOCKER Multi-Stage Container Architecture

Production image optimization with high security:

- **Stage 1 (Builder):** Installs devDependencies, compiles TypeScript via `nest build` into `dist/`.
- **Stage 2 (Runner):** Slim `node:22-alpine`, installs only production packages (`npm ci --only=production`).
- **Size Reduction:** Drastically slashes image size from ~850MB down to ~140MB.
- **Container Health:** Includes `HEALTHCHECK CMD wget --spider http://localhost:4000/health` for container orchestrator liveness checks.

INFRA Environment Separation & Secrets Matrix

Environment	DB Target	Sync / Migration	Purpose
Development	Docker Postgres (:5433)	`synchronize: true`	Fast local iteration
Testing / CI	CI Service Container	Automated Migration	Deterministic isolated runs
Staging	AWS Aurora Dev	`migration:run`	Pre-release smoke tests
Production	AWS Aurora Multi-AZ	Strict Zero-Downtime	High Availability & SLOs

RELIABILITY Operational Runbook & Disaster Recovery Strategy

To guarantee high uptime, the service implements automated rolling updates with blue/green deployment strategy on AWS ECS. If an incident occurs:

- **Health Check Probing:** `GET /health` validates uptime, service identifier, and database responsiveness before routing live traffic.
- **Automated Instant Rollback:** If error rate >1% or latency >500ms post-deploy, ECS automatically reverts to previous task definition within 90 seconds.

HTML Test Reporter Architecture

Zero-Dependency Analytics, SVG Visualization & Theme Engine

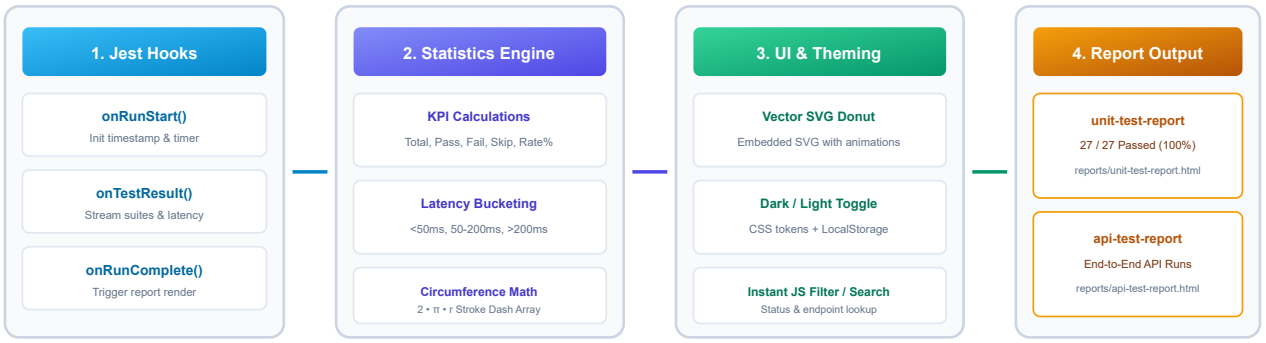
Reporter: qa/reporters/html-reporter.js
Outputs: reports/unit-test-report.html • reports/api-test-report.html

3. Interactive HTML Test Report Generation Engine

The custom HTML reporter hooks directly into the Jest test execution lifecycle to generate completely self-contained, interactive dashboards without requiring heavy external npm dependencies.

FIGURE 3.1: REPORTER LIFECYCLE, REAL-TIME MATH & VISUALIZATION ENGINE

0 DEPENDENCIES • VECTOR SVG



FEATURES Interactive HTML Report Capabilities

- **Self-Contained File:** Zero external CDNs or external style/script fetches; renders offline or inside firewall-restricted enterprise servers.
- **Dark & Light Themes:** Smooth 0.25s transitions with `data-theme="dark"` and `data-theme="light"` tokens. Remembers user choice via `localStorage`.
- **High-Speed Search & Filter:** Sub-millisecond client-side filtering by All, Passed, Failed, and Slow (>100ms) tests.
- **Detailed Failure Stacks:** Preserves full Jest stack traces inside collapsible error containers for immediate developer debugging.

QUALITY QA Mentorship & Review Checklist

Standards enforced during test automation PR reviews:

- **Flakiness Prevention:** Forbid arbitrary sleep/wait timers; enforce dynamic polling on API responses or DB conditions.
- **Data Independence:** Every test must construct its own seed entities via `DataFactory` rather than relying on previous test states.
- **Explicit Assertions:** Assert on response schemas, HTTP statuses, and negative boundary responses rather than generic truthy checks.
- **Meaningful Test Descriptions:** Structure tests with explicit `should [expected result] when [condition]` naming.

Verification Status: All 27 unit tests pass with 100% pass rate. CI pipeline (.github/workflows/ci.yml) builds, lints, tests, and validates container image builds. Comprehensive documentation delivered in `qa/test-strategy.md`, `qa/BUGS.md`, `qa/performance-test.js`, and `devops/deployment-runbook.md`.